Introducción a la Optimización de Consultas en PostgreSQL

Planificación, Algoritmos y Técnicas Avanzadas

Planificación Costo, estadísticas y EXPLAIN	**Algoritmos** Scans, Joins y pseudocódigos
Optimización Índices, configuración y reescritura	**Avanzado** CTEs, particionamiento y paginación

Contenido

01

Fundamentos de la Planificación de Consultas

Costo, Estadísticas y EXPLAIN

02

Algoritmos de Acceso a Datos (Scans)

Tipos de escaneo y sus pseudocódigos

03

Algoritmos de Unión (Joins)

Estrategias de join y sus pseudocódigos

04

Técnicas de Optimización Práctica

Índices, configuración y reescritura de consultas

05

Optimización Avanzada

CTEs, particionamiento y paginación eficiente

06

Herramientas y Errores Comunes

Diagnóstico y antipatrones frecuentes

El arte de encontrar el camino más rápido

Definición

Proceso de reescribir o estructurar consultas SQL para reducir el tiempo de ejecución y el consumo de recursos: **CPU, memoria e I/O**.

Objetivo

Que la base de datos encuentre los datos con el **menor costo posible**, garantizando escalabilidad y tiempos de respuesta predecibles.

La analogía del viaje

Planificar un viaje: la ruta más corta puede no ser la más rápida si hay tráfico. El optimizador de PostgreSQL busca la ruta con el menor **"costo" estimado**.

Tiempo de ejecución

Minimizar latencia de respuesta

Recursos

CPU, memoria e I/O optimizados

Escalabilidad

Comportamiento predecible bajo carga

¿Cómo piensa PostgreSQL?

1 Query Planner (Planificador)

Es el cerebro de PostgreSQL. Recibe la consulta SQL y genera un **"Plan de Ejecución"** óptimo.

2 Optimizador basado en costos (CBO)

PostgreSQL no busca la respuesta perfecta, busca la **más barata** basándose en estadísticas recopiladas.

Factores de decisión

Estadísticas

Cantidad de filas en las tablas

Índices disponibles

Estructuras de acceso rápido

I/O vs. CPU

Costo de leer disco vs. procesar en memoria

Proceso de planificación



Genera rutas

Calcula costos

Elige la menor

Estadísticas clave que PostgreSQL monitorea



n_distinct

Número estimado de valores distintos en una columna. Si es -1, significa que todos los valores son únicos. Si es positivo, es el número absoluto de valores distintos.



avg_width

Ancho promedio en bytes de cada valor en la columna. Afecta las decisiones sobre qué tipo de join usar (Hash Join vs. Nested Loop).



histogram_bounds

Límites de buckets para distribuciones de rango. Esencial para estimar cuántas filas caerán en un rango específico (ej: WHERE edad BETWEEN 20 AND 30).



null_frac

Fracción de valores NULL en la columna. Ayuda al planificador a estimar cuántas filas serán filtradas por condiciones IS NULL o IS NOT NULL.



most_common_vals

Lista de los valores más frecuentes en la columna. El planificador usa esto para estimar selectividad de condiciones WHERE.



reltuples y relpages

Número total de filas y páginas de disco ocupadas por la tabla. Determina si un Seq Scan o Index Scan es más eficiente.

¿Cómo se calcula el "costo"?

El costo es una **unidad abstracta** que combina I/O y CPU. Se configura en `postgresql.conf`.

Parámetros clave

Parámetro	Valor por defecto	Descripción
`seq_page_cost`	1.0	Costo de leer una página secuencialmente
`random_page_cost`	4.0	Costo de leer una página aleatoriamente (vía índice). **Ajustar a 1.1 en SSD**
`cpu_tuple_cost`	0.01	Costo de procesar una fila
`cpu_index_tuple_cost`	0.005	Costo de procesar una entrada de índice
`cpu_operator_cost`	0.0025	Costo de ejecutar un operador

Fórmula simplificada de costo



Ajustar `random_page_cost = 1.1` en servidores con SSD es una de las optimizaciones más impactantes y frecuentemente ignoradas.

Radiografiando nuestras consultas

EXPLAIN

Muestra el plan que PostgreSQL **planea ejecutar** sin ejecutar la consulta. Ideal para análisis previo sin costo de ejecución.

EXPLAIN ANALYZE

Ejecuta la consulta y muestra el plan real junto con los tiempos reales. Imprescindible para diagnóstico preciso.

Opciones avanzadas

ANALYZE

Tiempos reales

BUFFERS

Uso de caché (crucial)

VERBOSE

Detalle extendido

TIMING

Tiempos por nodo

Ejemplo básico

```
EXPLAIN ANALYZE
SELECT *
FROM usuarios
WHERE edad > 30;
```

Con opciones avanzadas

```
EXPLAIN (
 ANALYZE,
 BUFFERS,
 VERBOSE,
 TIMING
)
SELECT *
FROM usuarios
WHERE edad > 30;
```

Leyendo el EXPLAIN

El plan se lee **de adentro hacia afuera** y **de abajo hacia arriba**. Cada nodo representa una operación del motor.

Campo	Descripción	Notas clave
Nodos	Unidades de operación del plan	Leer de adentro hacia afuera, de abajo hacia arriba
Cost	`costo_inicial..costo_total`	Inicial = primera fila; Total = todas las filas
Rows	Número estimado de filas devueltas	Comparar con *actual rows* para detectar estimaciones erróneas
Width	Tamaño promedio en bytes de cada fila	Útil para estimar uso de memoria
Actual time	Tiempo real en milisegundos	Solo aparece con `ANALYZE`
Loops	Número de veces que se ejecuta el nodo	Crucial en Joins — multiplicar por el tiempo unitario

Truco clave: Si *rows estimadas* difiere mucho de *actual rows*, las estadísticas están desactualizadas. Ejecutar `VACUUM ANALYZE`.

La base de un buen plan: Estadísticas

PostgreSQL usa estadísticas para estimar cuántas filas devolverá una consulta. Si las estadísticas están **desactualizadas**, el planificador tomará malas decisiones.

Comando clave

```
VACUUM ANALYZE nombre_tabla;
```

Estadísticas extendidas

Para columnas correlacionadas (ej. `ciudad` y `estado`):

```
CREATE STATISTICS s1 (dependencies)
ON ciudad, estado
FROM direcciones;
```

Autovacuum

PostgreSQL ejecuta autovacuum automáticamente, pero en tablas muy activas puede quedarse corto.

Tablas activas

Ajustar umbrales de autovacuum para tablas con alta tasa de cambio

Columnas correlacionadas

Usar estadísticas extendidas para mejorar estimaciones en filtros combinados

Monitoreo

Consultar `pg_stat_user_tables` para detectar tablas sin ANALYZE reciente

¿Cómo lee PostgreSQL los datos? Scan Nodes

Definición: Los Scan Nodes son los **nodos hoja** del plan. Son los responsables de leer datos directamente de las tablas.

Seq Scan

Lee **toda la tabla** página por página. Eficiente para tablas pequeñas o cuando se necesita la mayoría de los datos.

Index Scan

Usa un índice para encontrar filas específicas. Eficiente con alta selectividad.

Index Only Scan

Obtiene **todos los datos del índice** sin acceder a la tabla. Requiere visibility map actualizado.

Bitmap Heap Scan

Usa un índice para crear un **mapa de bits** de páginas a leer. Combina ventajas de Seq e Index Scan.

TID Scan

Accede directamente a una fila por su **dirección física** (ctid). Caso de uso muy específico.

Factores de elección: Selectividad de los filtros, disponibilidad de índices y correlación física de los datos en disco.

Lectura completa de la tabla: Seq Scan

Descripción

Lee todas las páginas de la tabla, fila por fila. Es el método más simple y predecible del motor.

Pseudocódigo

```
for page in table.pages:
    for row in page.rows:
        if row.visible and \
           row.satisfies_filter():
            return row
```

Ejemplo en EXPLAIN

```
Seq Scan on tenk1
  (cost=0.00..458.00
   rows=10000
   width=244)
```

✓ Ventajas

Eficiente para **tablas pequeñas** o cuando se necesita la mayoría de los datos. Acceso secuencial al disco es rápido y predecible.

✗ Desventajas

Ineficiente en **tablas grandes con filtros selectivos**. Lee datos innecesarios cuando solo se necesita un subconjunto pequeño.

¿Cuándo aparece un Seq Scan?

Sin índice

No existe índice aplicable al filtro

Baja selectividad

El filtro devuelve >10% de las filas

Tabla pequeña

El costo del índice supera al scan completo

Index Scan

Búsqueda directa a través del índice

Busca en el índice (ej. B-tree) para encontrar las direcciones (TID) de las filas que coinciden, y luego accede a la tabla (heap) para obtener los datos.

Pseudocódigo

```
tids = index.search(filter_condition)  # Búsqueda en el índice
for tid in tids:
    page = table.fetch_page(tid.page_id)  # Acceso aleatorio a
la tabla
    row = page.get_row(tid.row_index)
    if row.visible:
        return row
```

Ejemplo en EXPLAIN

```
Index Scan using idx_age on usuarios
  (cost=0.29..8.30 rows=1 width=36)
  Index Cond: (edad = 30)
```

✓ Ventajas

Muy eficiente para consultas que devuelven **pocas filas** (alta selectividad).

✗ Desventajas

Costoso si devuelve **muchas filas** debido a los accesos aleatorios (I/O).



Es el método ideal para búsquedas puntuales (ej. `WHERE id = 5`).

Bitmap Heap Scan

El punto medio entre Seq Scan e Index Scan

Primero, un **Bitmap Index Scan** crea un mapa de bits en memoria con las páginas que contienen filas que coinciden. Luego, el **Bitmap Heap Scan** lee las páginas de la tabla en orden físico (secuencial), reduciendo el I/O aleatorio.

Pseudocódigo

```
bitmap = create_bitmap()  # Mapa de bits de páginas
for tid in index.search(filter_condition):
    bitmap.set_page(tid.page_id)

for page_id in bitmap.pages_in_order():  # Lectura secuencial
    page = table.read_page(page_id)
    for row in page.rows:
        if row.satisfies_filter():  # Re-check
            return row
```

Ejemplo en EXPLAIN

```
Bitmap Heap Scan on tenk1
  (cost=5.07..229.20 rows=101 width=244)
  Recheck Cond: (unique1 < 100)
  -> Bitmap Index Scan on tenk1_unique1
       (cost=0.00..5.04 rows=101 width=0)
       Index Cond: (unique1 < 100)
```

✓ Ventajas

Bueno para consultas con **selectividad media** (ej. devuelve 5–15% de la tabla).

✗ Desventajas

El paso de **"Recheck"** añade CPU. No es ideal para selectividades muy bajas o muy altas.

Es un algoritmo de dos fases que optimiza el acceso al disco convirtiendo lecturas aleatorias en secuenciales.

Index Only Scan

La consulta más rápida posible

Si todas las columnas necesarias para la consulta están en el índice, PostgreSQL **no necesita acceder a la tabla (heap)**. Responde directamente desde el índice.

Pseudocódigo

```
# El índice contiene (id, nombre)
for entry in index.search(filter_condition):
    return entry  # No se accede a la tabla
```

Ejemplo

```
-- Índice:
CREATE INDEX idx_nombre ON usuarios(nombre);

EXPLAIN ANALYZE
SELECT nombre FROM usuarios WHERE nombre = 'Ana';

-- Resultado:
Index Only Scan using idx_nombre on usuarios
```

Requisito

El índice debe **cubrir todas las columnas** del `SELECT` y del `WHERE`.

Visibility Map

PostgreSQL usa el **Visibility Map** para determinar si una página de la tabla es "all-visible" y no necesita ser verificada. Si no lo es, se produce un **"Heap Fetches"** (visible en EXPLAIN).



Es el "santo grial" de la optimización. Si ves un Index Only Scan con `Heap Fetches: 0`, es el plan más eficiente posible.

TID Scan

Acceso directo por dirección física

Accede a una fila directamente usando su **Tuple Identifier (TID)**, que consiste en un par `(page_id, row_index)`.

Uso

Se utiliza en consultas con `WHERE ctid = '(page_id,row_index)'` o en actualizaciones internas.

Ejemplo

```
EXPLAIN SELECT * FROM usuarios WHERE ctid = '(0,1)';

-- Resultado:
TID Scan on usuarios
```

Frecuencia

Poco común en consultas de aplicación.

Utilidad

Útil para comprender el funcionamiento interno de PostgreSQL.

Es poco común en consultas de aplicación, pero es útil entenderlo para comprender el funcionamiento interno.

Algoritmos de Unión

Joins en PostgreSQL

Estrategias que el planificador evalúa para combinar tablas de forma eficiente.

Nested Loop Join

Para cada fila de la tabla A, busca en la tabla B.

Hash Join

Crea una tabla hash en memoria de la tabla más pequeña y la compara con la grande.

Merge Join

Ordena ambas tablas y las combina secuencialmente.

Un **Join** combina filas de dos o más tablas basándose en una condición. La elección del planificador se basa en el tamaño de las tablas, índices disponibles, y si las entradas ya están ordenadas.

El planificador evalúa el costo de cada estrategia y elige la más económica. A veces, forzar un tipo de Join no es necesario si las estadísticas están al día.

Nested Loop Join

El más intuitivo, pero puede ser peligroso

Doble bucle. La tabla **externa (exterior)** se recorre fila por fila. Para cada fila, se busca en la tabla **interna (interior)** las que coinciden.

Pseudocódigo

```
for outer_row in outer_table:
    for inner_row in inner_table:
        if inner_row.key == outer_row.key:
            output(outer_row, inner_row)
```

Ejemplo en EXPLAIN

```
Nested Loop Join
  (cost=0.29..115.32 rows=1000 width=44)
  -> Seq Scan on users u
       (cost=0.00..22.00 rows=1000 width=36)
  -> Index Scan using orders_user_id_idx on orders o
       (cost=0.29..0.49 rows=1 width=16)
       Index Cond: (user_id = u.user_id)
```

Optimización clave

Si la tabla interna tiene un **índice en la clave de unión**, la búsqueda interna se convierte en un Index Scan, reduciendo la complejidad de `O(N×M)` a `O(N log M)`.

✓ Ideal para

Conjuntos de datos pequeños o cuando la tabla interna tiene un índice eficiente. Algoritmo preferido para **OLTP** (búsquedas puntuales).

⚠ Peligroso cuando

No hay índice en la tabla interna: complejidad `O(N×M)`.

Es excelente para conjuntos de datos pequeños o cuando la tabla interna tiene un índice eficiente. Es el algoritmo preferido para OLTP.

Hash Join

El rey de las uniones de igualdad

Primero, se escanea la tabla más pequeña y se construye una **tabla hash en memoria**. Luego, se escanea la tabla más grande y se **busca en** la tabla hash para encontrar coincidencias.

Pseudocódigo

```
hash_table = build_hash_table(smaller_table)
for row in larger_table:
    if row.key in hash_table:
        output(row, hash_table[row.key])
```

Ejemplo en EXPLAIN

```
Hash Join
  (cost=10.38..139.25 rows=4079 width=20)
  Hash Cond: (city.countrycode = country.code)
  -> Seq Scan on city
       (cost=0.00..72.79 rows=4079 width=13)
  -> Hash (cost=7.39..7.39 rows=239 width=15)
       -> Seq Scan on country
            (cost=0.00..7.39 rows=239 width=15)
```

Requisito

Solo funciona para **uniones de igualdad** (`=`).

✓ Ventajas

Muy eficiente para unir **tablas grandes**. El planificador construye la hash en la tabla más pequeña para optimizar memoria.

✗ Desventajas

Consume memoria. No sirve para uniones no-equi (`<`, `>`, `BETWEEN`).

Es muy eficiente para unir tablas grandes. El planificador elige construir la tabla hash en la tabla más pequeña para optimizar el uso de memoria.

Merge Join

El poder del orden

Ordena ambas tablas por la clave de unión y luego las recorre **simultáneamente**, comparando las claves y emitiendo las coincidencias.

Pseudocódigo

```
sort(left_table, key)
sort(right_table, key)
while left_pointer and right_pointer:
    if left.key == right.key:
        output(left, right)
        advance both
    elif left.key < right.key:
        advance left
    else:
        advance right
```

Ejemplo en EXPLAIN

```
Merge Join (cost=...)
  Merge Cond: (a.id = b.id)
  -> Sort ...
       -> Seq Scan on a ...
  -> Sort ...
       -> Seq Scan on b ...
```

✓ Ventajas

No requiere memoria adicional (a diferencia del Hash Join) y produce resultados ordenados.

✗ Desventajas

Requiere que ambas entradas estén **ordenadas**. Si no lo están, el costo de la ordenación es alto.



Nota del orador: Ideal para uniones de rango (`<`, `>`, `<=`, `>=`) o cuando ambos lados ya están ordenados (ej. por un índice).

Comparativa de Algoritmos de Join

¿Cuándo usar cada uno?

Algoritmo	Cuándo usarlo	Ventaja	Desventaja
Nested Loop	Tabla externa pequeña, tabla interna con índice en la clave de unión.	Rápido para subconjuntos pequeños. Permite "early exit" (ej. con `LIMIT`).	`O(N×M)` si no hay índice.
Hash Join	Uniones de igualdad. Una tabla es significativamente más pequeña que la otra.	Muy rápido para grandes volúmenes de datos.	Consume memoria. No sirve para uniones no-equi.
Merge Join	Ambas tablas ya están ordenadas o el resultado debe estar ordenado.	No consume memoria. Produce resultados ordenados.	La ordenación previa puede ser costosa.



La decisión del planificador depende del costo estimado. Si las estadísticas están desactualizadas, puede elegir un algoritmo subóptimo (ej. Nested Loop cuando un Hash Join sería mejor).

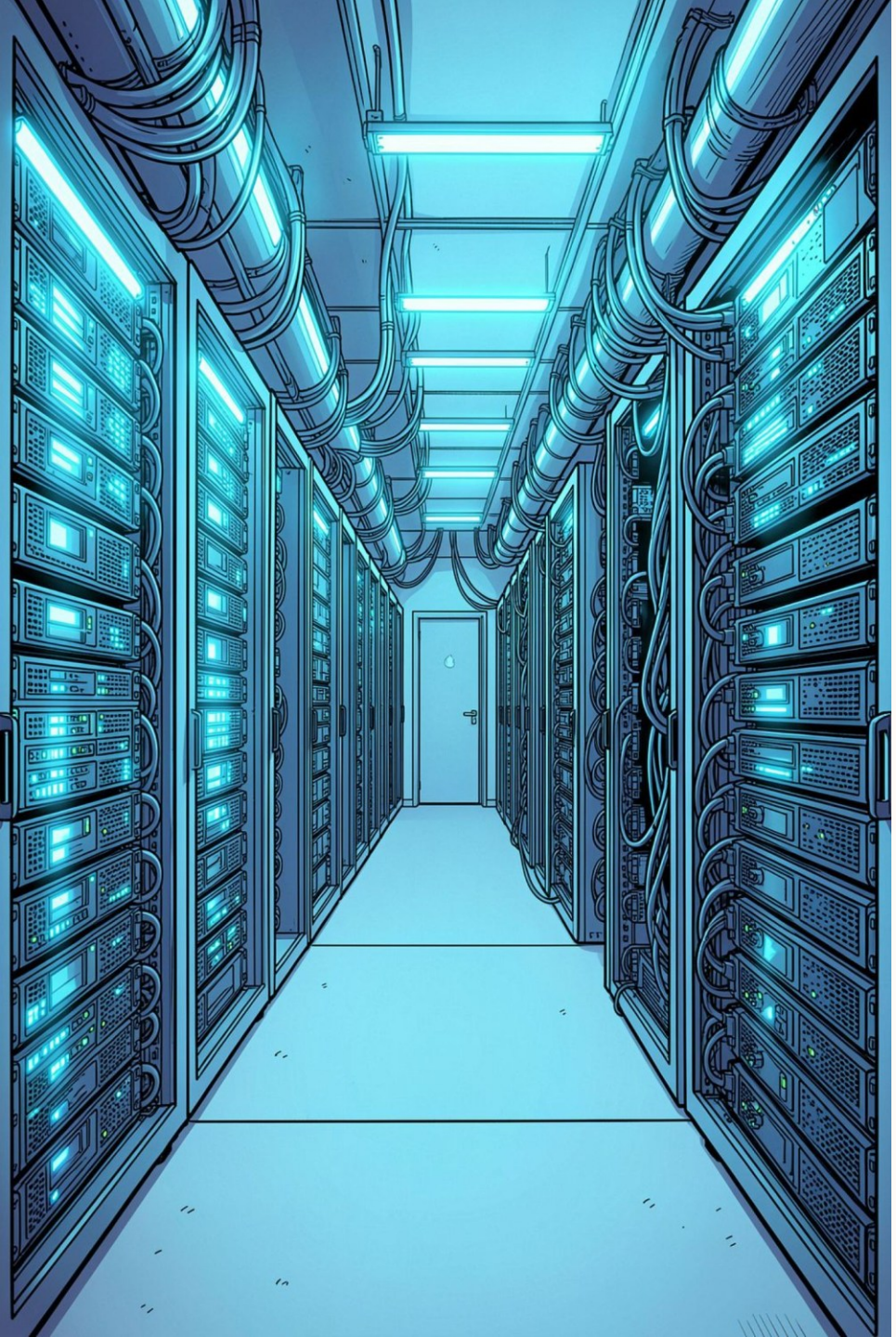
Resumen: Algoritmos de Escaneo y de Joins

Guía rápida de referencia

Algoritmo	Cuándo el planificador lo elige	Clave a recordar
Index Scan	Alta selectividad, pocas filas devueltas.	Acceso aleatorio al heap. Ideal para `WHERE id = 5`.
Bitmap Heap Scan	Selectividad media (5–15% de la tabla).	Convierte I/O aleatorio en secuencial. Dos fases: bitmap + heap.
Index Only Scan	Todas las columnas cubiertas por el índice.	"Santo grial". `Heap Fetches: 0` = plan óptimo.
TID Scan	Consulta con `WHERE ctid = '(...)'`.	Acceso directo por dirección física. Poco común.
Nested Loop	Tabla interna pequeña con índice. OLTP.	`O(N log M)` con índice. Soporta "early exit".
Hash Join	Uniones de igualdad con tablas grandes.	Construye hash en la tabla más pequeña. Consume memoria.
Merge Join	Entradas ya ordenadas o uniones de rango.	Sin memoria extra. Produce resultados ordenados.



💡 El planificador de PostgreSQL elige automáticamente el algoritmo de menor costo estimado. Mantener las estadísticas actualizadas con `ANALYZE` es fundamental para decisiones óptimas.



El arma más poderosa (pero peligrosa)

Tipos de Índices y su uso

B-tree

Por defecto. Ideal para `=`, `<`, `>`, `BETWEEN`, `IN`, `LIKE 'texto%'`.

GIN

Ideal para tipos de datos compuestos (ej. `jsonb`, `array`, `tsvector`). Consultas de contención (`@>`, `<@`).

GiST

Ideal para datos geométricos y de rango. Soporta búsquedas de "vecino más cercano".

HASH

Calcula un hash del valor buscado y va directo al bucket correspondiente. Complejidad O(1). Compacto.

BRIN

Ideal para tablas muy grandes con correlación física (ej. `created_at`). El índice es minúsculo.

Costo de los índices

Ralentizan las escrituras (`INSERT`, `UPDATE`, `DELETE`).

Ejemplo

```
CREATE INDEX idx_usuarios_edad
  ON usuarios(edad);

CREATE INDEX idx_logs_fecha
  ON logs USING BRIN (fecha_creacion);

CREATE INDEX idx_documentos_tags
  ON documentos USING GIN (tags);
```

⚠️ No creen índices a ciegas. Cada índice añade sobrecarga. Usen `pg_stat_user_indexes` para encontrar índices no utilizados.

Mejores prácticas en el día a día

Evita SELECT *

Solo trae las columnas que necesitas. Reduce el uso de memoria y red.

Usa LIMIT

Si solo necesitas los primeros 10 resultados, no traigas 10,000.

UNION ALL vs UNION

`UNION ALL` es más rápido porque no elimina duplicados.

Cuidado con las funciones en el WHERE

El error más común es aplicar funciones a columnas indexadas en el `WHERE`. Eso obliga a PostgreSQL a hacer un `Seq Scan`.

Ajustando el motor

shared_buffers

Memoria para caché de datos.
Recomendado: 25% de la RAM.

effective_cache_size

Estimación de la memoria caché total (SO + PostgreSQL).
Recomendado: 50–75% de la RAM.

work_mem

Memoria para operaciones de ordenación y hash. Aumentar si hay muchos `Sort` en disco.

maintenance_work_mem

Memoria para `VACUUM`, `CREATE INDEX`, etc.

random_page_cost

Ajustar a `1.1` en SSD/NVMe.

max_parallel_workers_per_gather

Número de workers para consultas paralelas.



Estos parámetros son globales. Ajusten con cuidado y monitoreen el impacto.

¿Optimización o barrera?

Common Table Expressions (CTEs)

PostgreSQL < 12

1

Las CTEs eran una **"barrera de optimización"**. Se materializaban y no se podían fusionar con la consulta principal.

2

PostgreSQL 12+

Por defecto, las CTEs se pueden **"inlinear"** (fusionar) si se usan una sola vez. Esto permite que los índices se usen dentro de la CTE.

Materialización Forzada

3

Usa `WITH nombre AS MATERIALIZED (...)` para forzar el comportamiento antiguo cuando sea necesario.

Ejemplo

```
-- PostgreSQL 12+ optimizará esto
WITH shipped_orders AS (
  SELECT *
  FROM orders
  WHERE status = 'shipped'
)
SELECT *
FROM shipped_orders
JOIN order_items ON ...
```

En versiones modernas, las CTEs ya no son un problema. Pero en versiones antiguas, podían causar cuellos de botella severos.

Dividir para reinar

Particionamiento de tablas

Definición

Dividir una tabla grande en tablas más pequeñas (particiones) lógicamente.

Ventajas

Mejora el mantenimiento (ej. `DROP` de una partición antigua) y la velocidad de escaneo (*partition pruning*).

Tipos

El particionamiento es esencial para tablas que superan los ~100M de filas. Permite que las consultas solo escaneen las particiones relevantes.

Ejemplo

```
CREATE TABLE orders (
  order_id      BIGSERIAL,
  order_date    DATE NOT NULL,
  customer_id   BIGINT,
  total_amount  NUMERIC
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_202301
  PARTITION OF orders
  FOR VALUES FROM ('2023-01-01')
            TO   ('2023-02-01');
```

RANGE	LIST	HASH



El adiós al OFFSET

Problema del OFFSET

`OFFSET 100000` obliga a PostgreSQL a leer y descartar 100,000 filas. Escala **linealmente** con el número de página.

Solución: Paginación por Keyset (cursor)

Usa el valor de la última fila vista para obtener la siguiente página. Rendimiento **constante** sin importar la profundidad.

> Requiere una ordenación total estable (ej. por fecha + id). Es la única forma de paginar eficientemente en tablas muy grandes.

```
--Ejemplo:

-- Paginación OFFSET (lenta)
SELECT * FROM orders ORDER BY order_date DESC
LIMIT 10 OFFSET 100000;

-- Paginación Keyset (rápida y constante)
-- Suponiendo que el último cursor fue:
   order_date = '2023-01-15', id = 12345
SELECT * FROM orders
WHERE (order_date, id) < ('2023-01-15', 12345)
ORDER BY order_date DESC, id DESC LIMIT 10;
```

El detector de consultas lentas

Extensión: `pg_stat_statements`



Debe añadirse a `shared_preload_libraries`. Acumula estadísticas de todas las consultas ejecutadas: tiempo, llamadas, I/O.

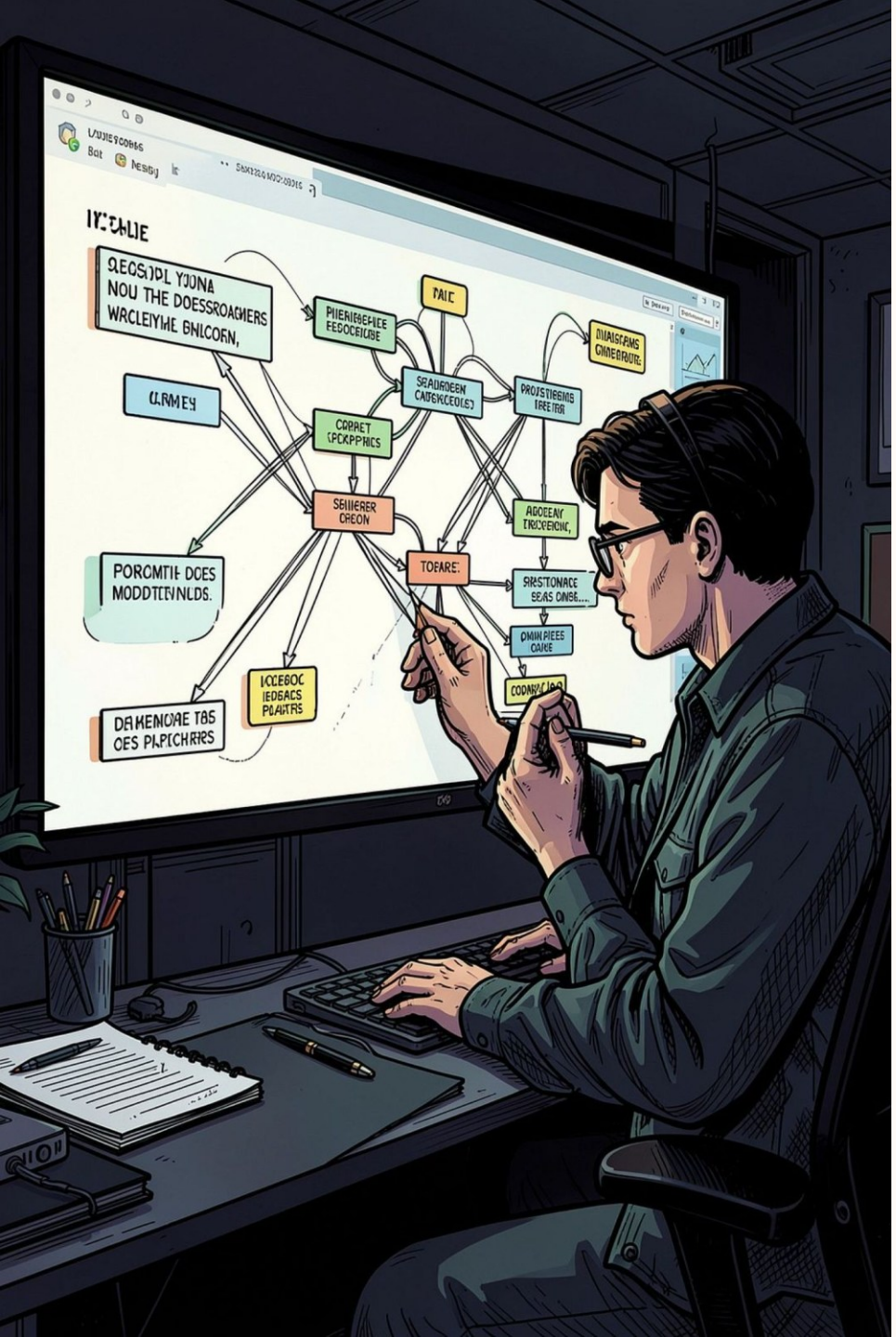
Consulta útil: top 10 por tiempo total

```
SELECT
  queryid,
  query,
  calls,
  total_exec_time,
  mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Nuevo en PG 19

Contadores para planes genéricos vs. personalizados: `generic_plan_calls`, `custom_plan_calls`.

No optimicen a ciegas. Usen `pg_stat_statements` para encontrar los verdaderos cuellos de botella.



Entendiendo el plan de forma visual

explain.depesz.com

Pega el resultado de `EXPLAIN ANALYZE` y obtén un análisis visual detallado.

explain.dalibo.com

Similar a depesz, con visualización de árbol y análisis de *hotspots*.

pgAdmin / DBeaver

Tienen analizadores visuales de planes incorporados directamente en la interfaz.

AutoEXPLAIN

En psql, ejecuta `\timing` y `EXPLAIN ANALYZE` rápidamente para diagnóstico inmediato.

Estas herramientas son excelentes para identificar rápidamente los nodos más costosos.

¿Qué NO hacer?

⊘ Demasiados índices

Cada índice ralentiza las operaciones de escritura (`INSERT`, `UPDATE`, `DELETE`).

⊘ OFFSET gigantes

`OFFSET 100000` obliga a PostgreSQL a leer y descartar 100,000 filas.

⊘ Uso excesivo de DISTINCT

Suele indicar un problema en el diseño de las consultas o Joins innecesarios.

⊘ No usar EXPLAIN

Optimizar a ciegas es el peor error que puedes cometer.

⊘ Ignorar las estadísticas

Un `VACUUM ANALYZE` regular es fundamental para mantener el planificador informado.

La optimización no es solo agregar índices. A veces, menos es más. La reescritura de consultas es a menudo más efectiva.

Análisis y solución

Caso práctico: de 2 segundos a 2 ms

01

Problema

`SELECT * FROM pedidos WHERE cliente_id = 12345;` tarda **2 segundos**.

02

Diagnóstico

`EXPLAIN ANALYZE` muestra un `Seq Scan on pedidos` con `Rows Removed by Filter: 1,000,000`.

03

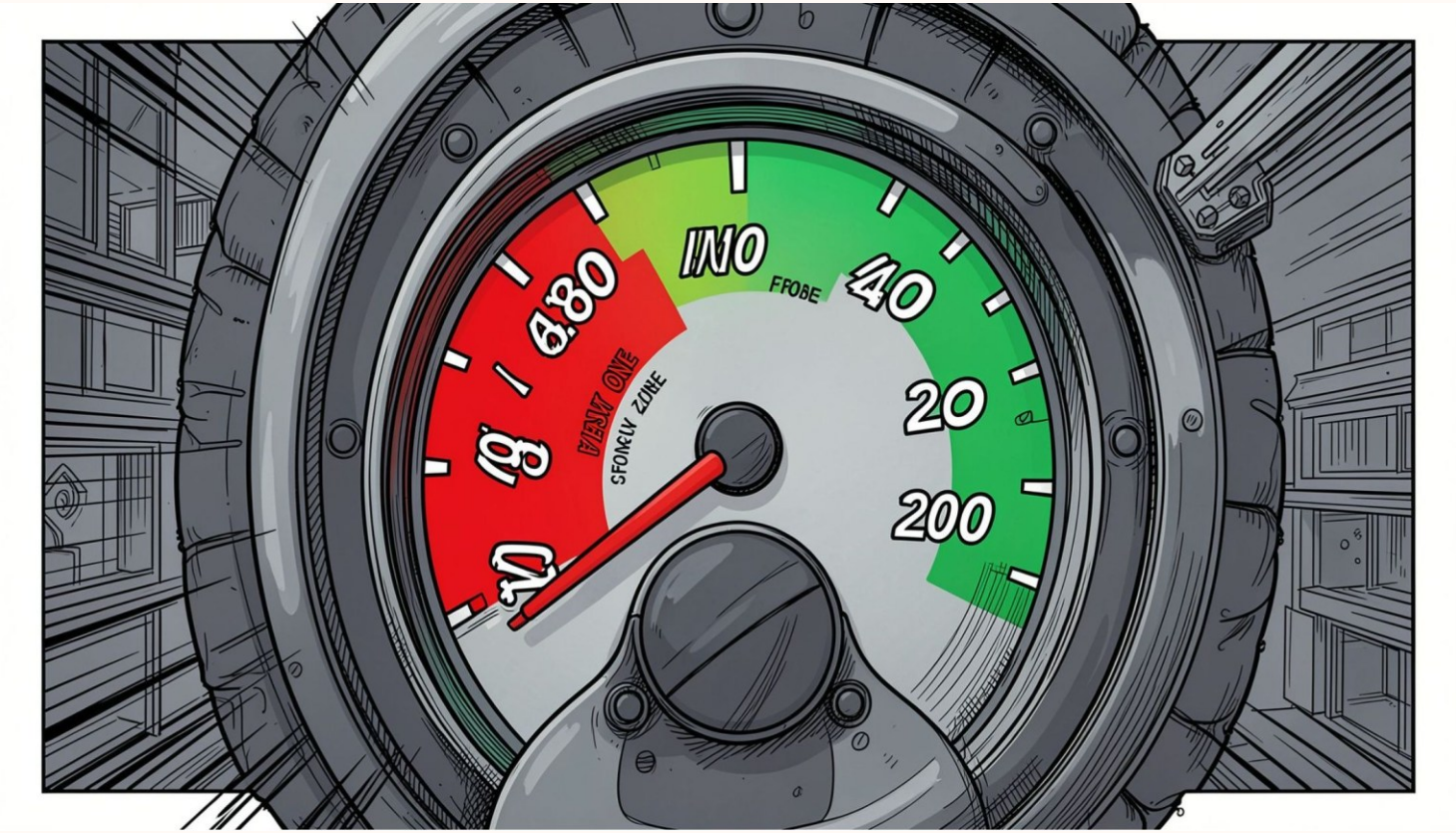
Causa

No hay índice en `cliente_id`.

04

Solución

```
CREATE INDEX idx_pedidos_cliente
  ON pedidos(cliente_id);
```



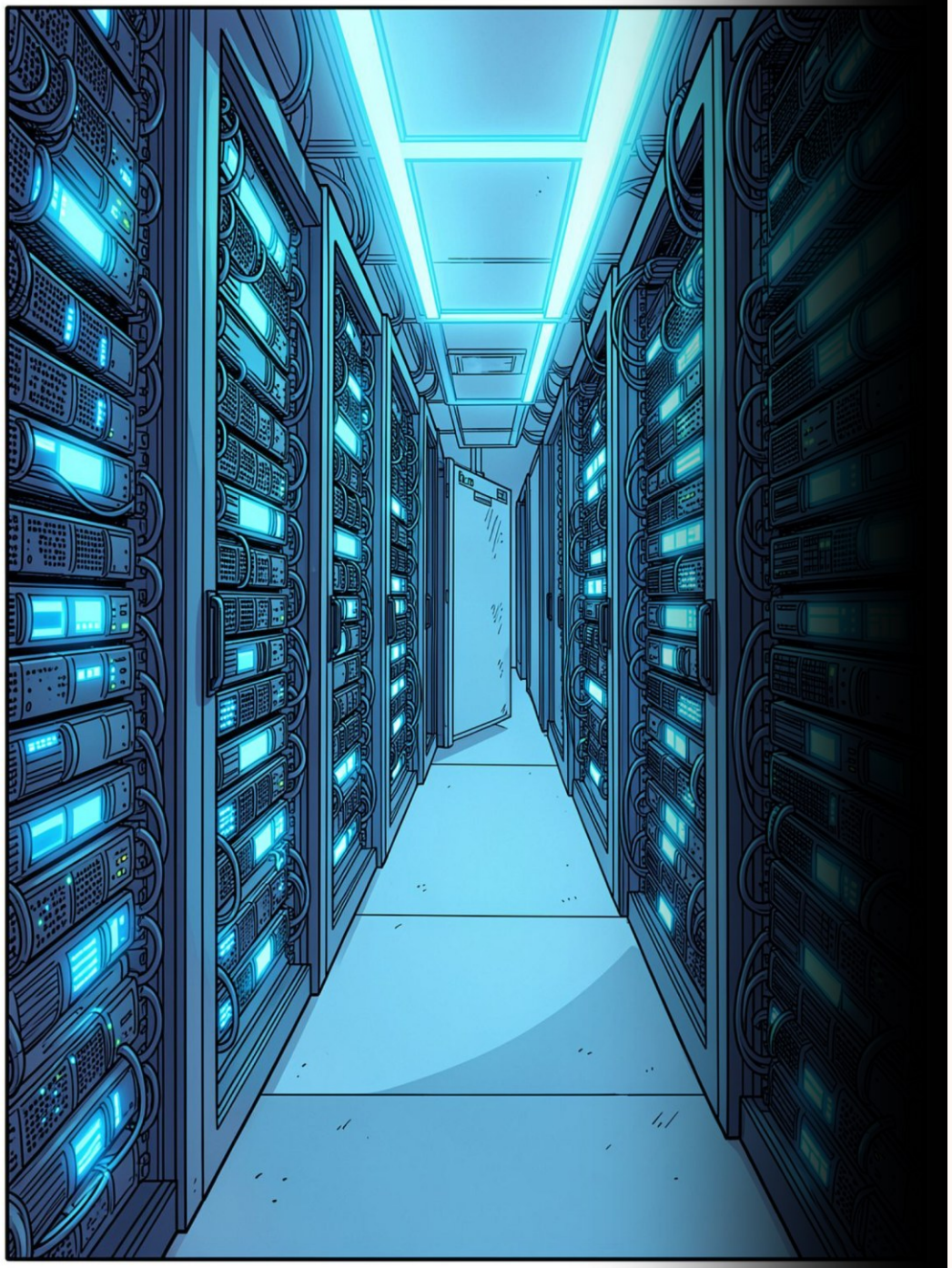
Antes

Seq Scan — **2,000 ms**

Después

Index Scan — **2 ms**

Este es el caso más común. Un índice bien colocado puede transformar una consulta.



Análisis y solución

El Problema

Consulta con JOIN entre ventas (10M filas) y productos (1M filas) tarda **30 segundos**.

Diagnóstico

EXPLAIN ANALYZE muestra un **Nested Loop Join** con un Seq Scan en la tabla interna.

Causa

El planificador eligió Nested Loop porque creyó que la tabla interna devolvería pocas filas, pero no fue así.

Solución

Actualizar estadísticas con `ANALYZE` o reescribir la consulta. Para diagnóstico, forzar un Hash Join:

```
SET enable_nestloop = off;
```

Resultado

El plan cambia a **Hash Join** y la consulta tarda **1.5 segundos** — una mejora de 20×.

Análisis y solución

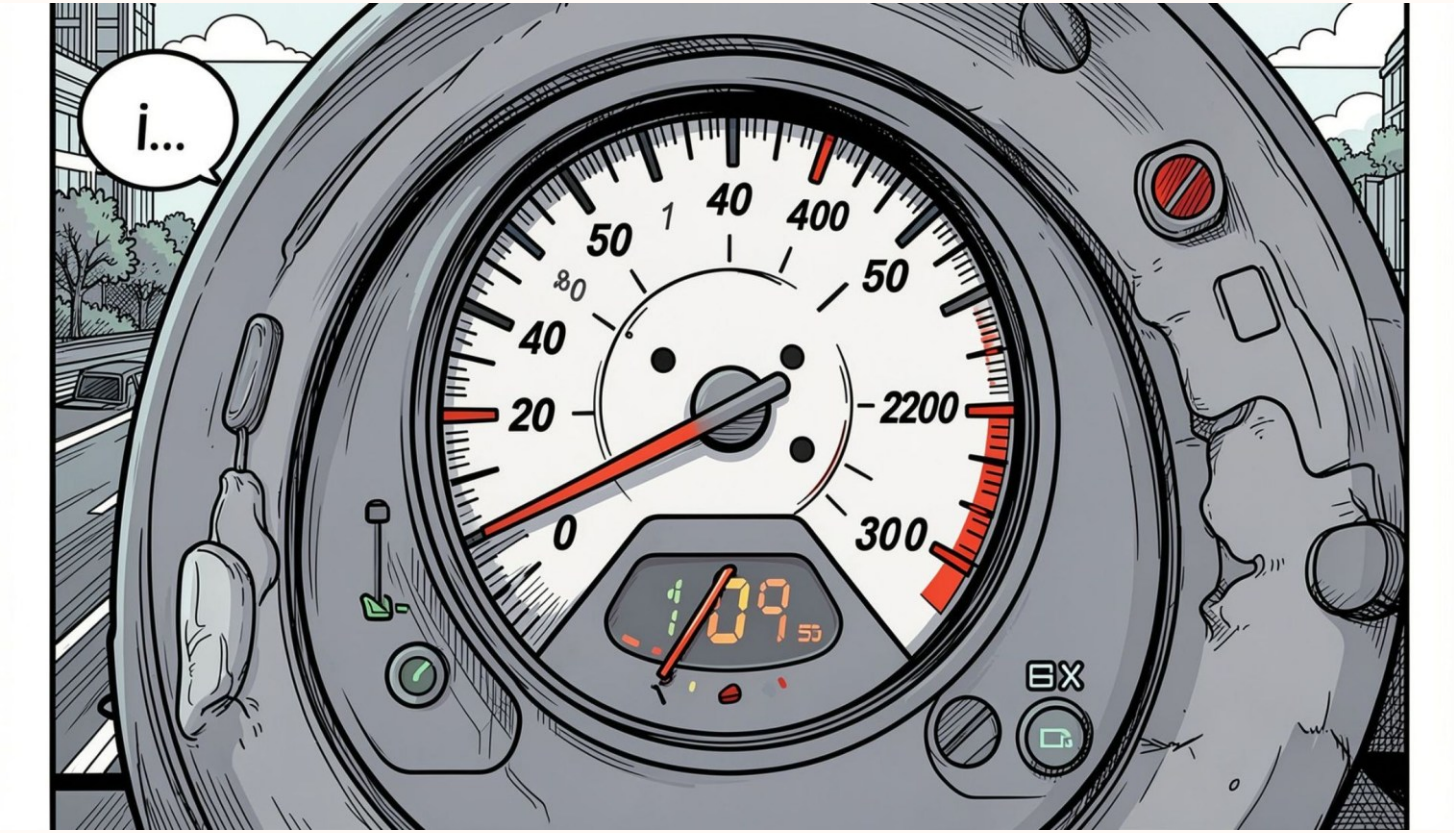
El Problema

```
SELECT * FROM logs
ORDER BY fecha DESC
LIMIT 10 OFFSET 500000;
```

Tarda **5 segundos**. El `OFFSET` obliga a leer 500,000 filas y descartarlas.

Solución: Paginación por Keyset

```
SELECT * FROM logs
WHERE (fecha, id) < ('2023-01-15
10:00:00', 12345)
ORDER BY fecha DESC, id DESC
LIMIT 10;
```



Resultado

La consulta tarda **5 ms**, independientemente de la página. Una mejora de **1000×**.



El trabajo continuo

VACUUM

Recupera espacio de filas muertas.

ANALYZE

Actualiza estadísticas del planificador.

VACUUM ANALYZE

Ambas operaciones en un solo comando.

Autovacuum

Configurar `autovacuum_vacuum_scale_factor` y `autovacuum_analyze_scale_factor` para tablas de alto movimiento.

Monitoreo — Ver tablas que necesitan mantenimiento:

```
SELECT relname, last_vacuum, last_analyze, n_dead_tup
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

Checklist de Optimización

1	**Analizar** Usa `EXPLAIN ANALYZE` para entender el plan actual.
2	**Estadísticas** Mantén las estadísticas actualizadas con `ANALYZE`.
3	**Índices** Crea índices en columnas de filtro y join. Evita índices innecesarios.
4	**Reescritura** Evita funciones en WHERE, usa UNION ALL, evita SELECT *.
5	**Configuración** Ajusta `work_mem`, `shared_buffers`, `random_page_cost`.
6	**Avanzado** Considera particionamiento, CTEs materializadas, paginación por keyset.
7	**Monitoreo** Usa `pg_stat_statements` para encontrar consultas problemáticas.

Lo que debes evitar



Optimizar sin medir

"Creo que esto será más rápido". Usa siempre `EXPLAIN ANALYZE`.

Asumir que los índices siempre son buenos

Un índice en una columna con baja cardinalidad (ej. `genero`) puede no ser útil.

Ignorar el impacto en escrituras

Cada índice ralentiza los `INSERT`/`UPDATE`.

No probar en producción

Los planes pueden variar entre entornos. Prueba en un entorno similar a producción.

Olvidar el mantenimiento

Un `VACUUM` regular es crucial.

Estrategias para múltiples tablas



Orden de Join

El planificador decide el orden. A veces, forzar un orden con JOIN explícito o subconsultas puede ayudar.



Estadísticas Extendidas

Para columnas correlacionadas en múltiples tablas, usa estadísticas extendidas para mejorar las estimaciones del planificador.



pg_hint_plan

Extensión para forzar planes específicos. Usar con precaución — es una herramienta de último recurso.

Ejemplo — Forzar un orden de join:

```
-- Consulta original (puede generar un mal plan)
SELECT * FROM a JOIN b ON a.id = b.a_id JOIN c ON b.id = c.b_id;

-- Forzar un orden de join
SELECT * FROM (SELECT * FROM a JOIN b ON a.id = b.a_id) AS ab
JOIN c ON ab.id = c.b_id;
```

El poder de las funciones de ventana

Definición

`ROW_NUMBER()`, `RANK()`, `SUM() OVER (...)` — calculan valores sobre un conjunto de filas relacionadas con la fila actual.

Cuidado

Las window functions pueden ser costosas si no se usan correctamente.

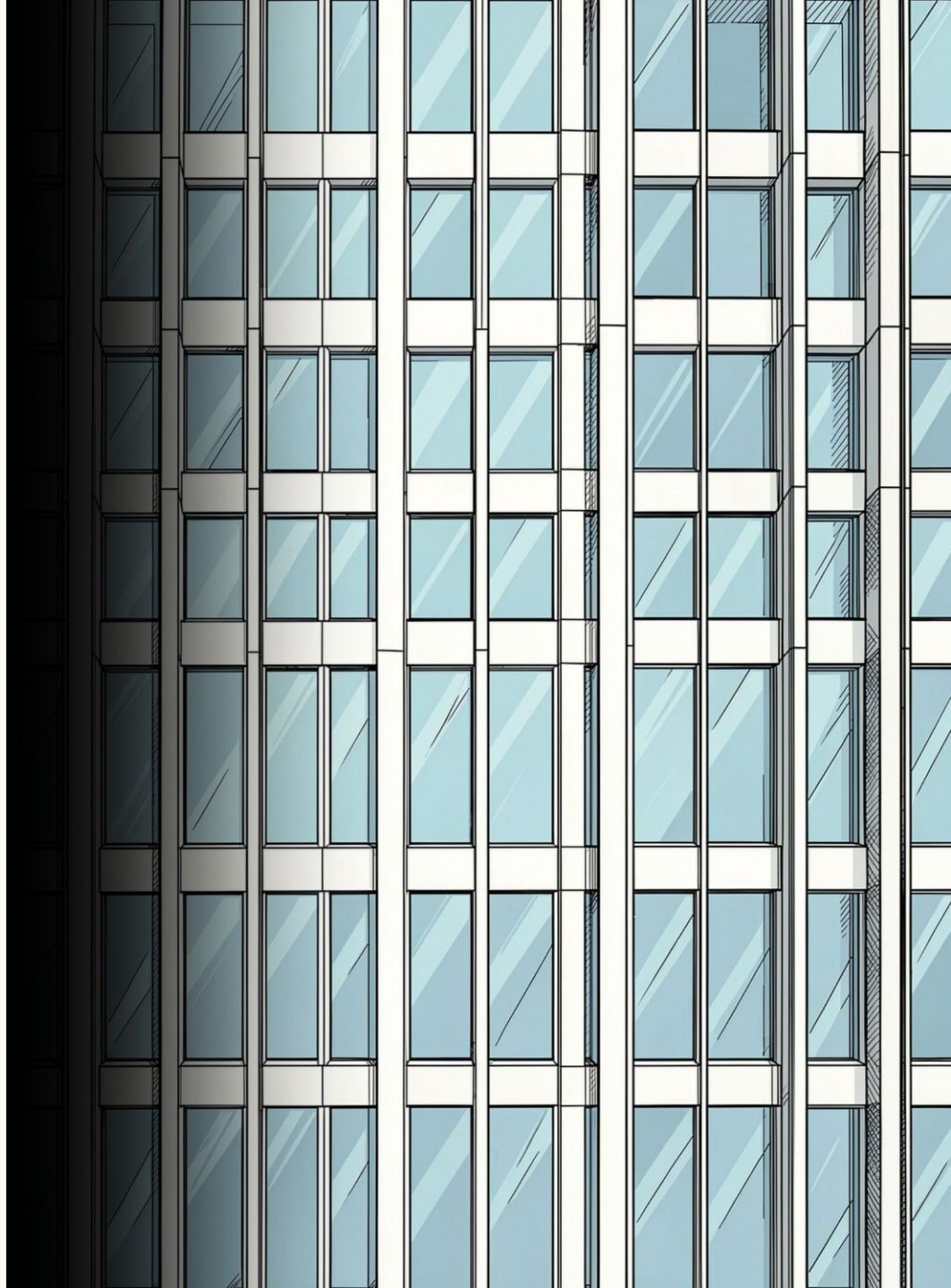
Optimización

Asegúrate de que el `PARTITION BY` y `ORDER BY` estén indexados si es posible.

Ejemplo

```
-- Esta consulta puede ser lenta
si no hay
-- índice en (departamento,
salario)
SELECT
  nombre,
  salario,
  RANK() OVER (
    PARTITION BY departamento
    ORDER BY salario DESC
  ) as rango
FROM empleados;
```

⚠️ Crea un índice compuesto en `(departamento, salario)` para acelerar esta consulta.



No solo las lecturas son importantes

Batch Inserts

Usa `INSERT INTO ... VALUES (...), (...)` en lugar de múltiples INSERT individuales.

COPY

Para cargas masivas, `COPY` es mucho más rápido que `INSERT`.

Índices

Elimina índices innecesarios que ralentizan las escrituras.

fillfactor

Ajusta el `fillfactor` para reservar espacio para actualizaciones (HOT updates).

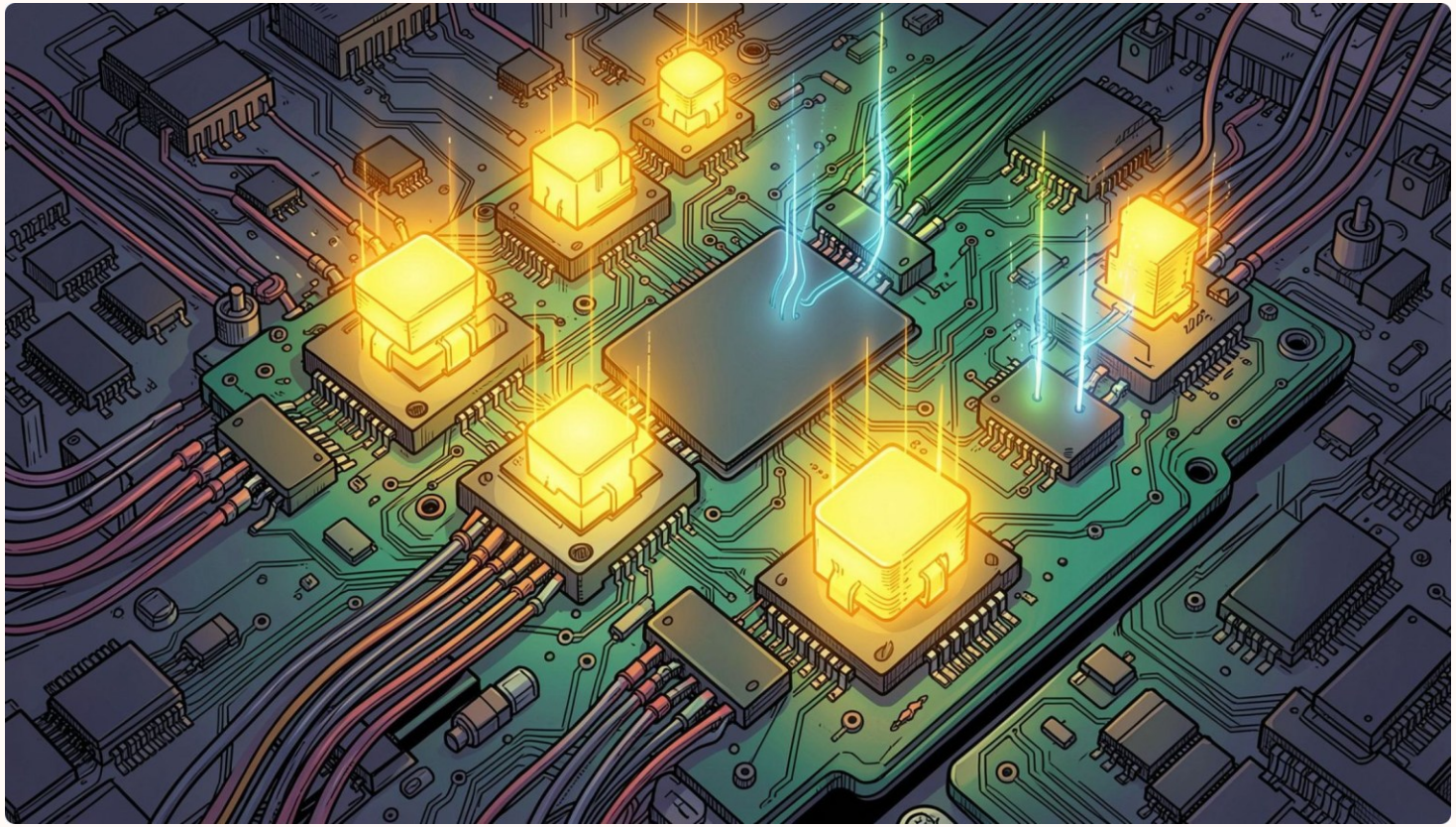
Ejemplo — Batch Insert

```
-- En lugar de:
INSERT INTO logs (mensaje) VALUES ('a');
INSERT INTO logs (mensaje) VALUES ('b');

-- Usa:
INSERT INTO logs (mensaje)
VALUES ('a'), ('b');
```

Un solo `INSERT` con múltiples valores reduce el overhead de red y transacciones significativamente.

Aprovechando múltiples núcleos



Paralelismo en PostgreSQL

PostgreSQL puede paralelizar escaneos, joins y agregaciones para aprovechar múltiples núcleos del servidor.

Parámetros clave

- `max_parallel_workers_per_gather` — workers por nodo Gather
- `max_worker_processes` — total de workers del sistema

Señal en EXPLAIN

Busca `Gather` o `Gather Merge` en el plan de ejecución:

```
Gather (cost=1000.00..1500.00 rows=10000
width=4)
 Workers Planned: 2
 -> Parallel Seq Scan on large_table
 (cost=0.00..500.00 rows=5000 width=4)
```

Puntos clave para llevar

La optimización es un proceso iterativo

No es un evento único — mide, ajusta y repite continuamente.

Entiende cómo PostgreSQL planifica

Usa `EXPLAIN ANALYZE` para interpretar cada plan de ejecución.

Mantén las estadísticas actualizadas

Ejecuta `VACUUM ANALYZE` regularmente en tablas de alto movimiento.

Escribe consultas limpias

Evita `SELECT *` y funciones en columnas indexadas.

Conoce los algoritmos de Scan y Join

Te ayudará a interpretar los planes y tomar mejores decisiones.

Mide, no adivines

Los datos son tu única fuente de verdad en optimización.